Analysis of REST API Performance Using Different Authentication Mechanisms

Author - Kartik
Co-author – Dr. Shikha Tuteja [Associate Professor]

Chitkara University Institute of Engineering & Technology, Rajpura [140401] Punjab, India
Computer Science & Engineering

kartik1752.be22@chitkara.edu.in

Shikha.1290@chitkara.edu.in

Abstract. Representational State Transfer (REST) APIs form the backbone of modern web and mobile systems, where authentication directly impacts both security and performance. This study presents a comparative evaluation of three widely used authentication mechanisms—API Key, JSON Web Token (JWT), and OAuth 2.0—using a Node.js-based REST service and Apache JMeter under controlled workloads. Key performance metrics, including response time, latency, throughput, CPU utilization, and memory usage, were analyzed. The results indicate that API Key authentication achieves the lowest response time (~118 ms), lowest latency (~41 ms), and highest throughput (~842 req/s) due to minimal processing overhead. JWT introduces moderate overhead (~176 ms response time) as a result of token parsing and signature verification, while OAuth 2.0 exhibits the highest latency (~88 ms), lowest throughput (~533 req/s), and greater resource consumption due to multi-step token validation and enhanced security controls. These findings demonstrate a clear security–performance tradeoff, where stronger authentication mechanisms incur higher computational cost. The study provides practical guidance for selecting appropriate authentication strategies based on application requirements, scalability, and security considerations.

1 **Keywords:** REST API, Authentication, API Key, JWT, OAuth 2.0, Performance Evaluation.

1 Introduction

1.1 Overview Of REST APIs

REST APIs are now a foundational integration pattern for distributed software systems because they provide a lightweight, platform-independent, and scalable way to expose application functionality over HTTP. Their stateless request–response model allows clients and servers to evolve independently, which is one reason REST APIs are extensively used in cloud platforms, microservice architectures, mobile backends, and third-party developer ecosystems. However, this openness also introduces security risks. APIs often expose sensitive business data, account information, payment

interfaces, and administrative operations, making authentication a mandatory design layer rather than an optional enhancement.

Authentication in REST APIs verifies who is making a request before any protected operation is processed. In practical deployments, the mechanism selected for authentication can affect not only security posture but also response characteristics perceived by end users. A strong mechanism that introduces significant computational overhead can reduce throughput and increase latency under load, whereas a simpler mechanism can improve speed but may provide weaker security guarantees. This creates a fundamental security–performance tradeoff that engineering teams must evaluate carefully.

In real-world systems, this tradeoff is visible in many scenarios. Public APIs consumed by low-risk integrations may prioritize simplicity and speed, while fintech, healthcare, and enterprise collaboration systems usually require stronger access control, delegated authorization, and token lifecycle management, even at the cost of increased processing overhead. Because organizations frequently scale APIs to serve thousands of concurrent users, understanding the runtime behavior of authentication methods is essential for architecture decisions. This paper therefore compares API Key, JWT, and OAuth 2.0 mechanisms through controlled benchmarking and interprets the findings from both performance and practical deployment perspectives.

2 Related Work

2.1 API Key Authentication Studies

Prior research has established that API security mechanisms introduce measurable overhead in web service environments, especially under concurrent workloads. Fielding's architectural principles for REST emphasized scalability and stateless communication, but security concerns have become more prominent as API adoption has grown [1]. Several studies report that lightweight token or key-based methods reduce per-request processing cost, while cryptographic validation and delegated authorization flows increase latency [2], [3].

2.2 JWT-Based Authentication

API Key-based authentication is often discussed as a minimal-access control mechanism suitable for service identification and quota management, but it is criticized for limited granularity and weak replay resistance when not combined with transport-level protections [4]. JWT-based approaches have received extensive attention because they preserve REST statelessness and carry claims that can be verified locally, reducing frequent database lookups. Even so, signature verification, claim parsing, and token refresh strategies can add moderate processing time as load increases [5].

2.3 OAuth 2.0 Approaches

OAuth 2.0 has been widely adopted in enterprise and third-party integration ecosystems due to delegated authorization and scope-based access control. Existing performance analyses indicate that OAuth-based flows incur higher setup and validation overhead, especially when introspection endpoints, token revocation checks, or external authorization servers are involved [6], [7]. Contemporary API security frameworks further recommend combining OAuth 2.0 with short-lived tokens and layered monitoring to balance usability and risk [8].

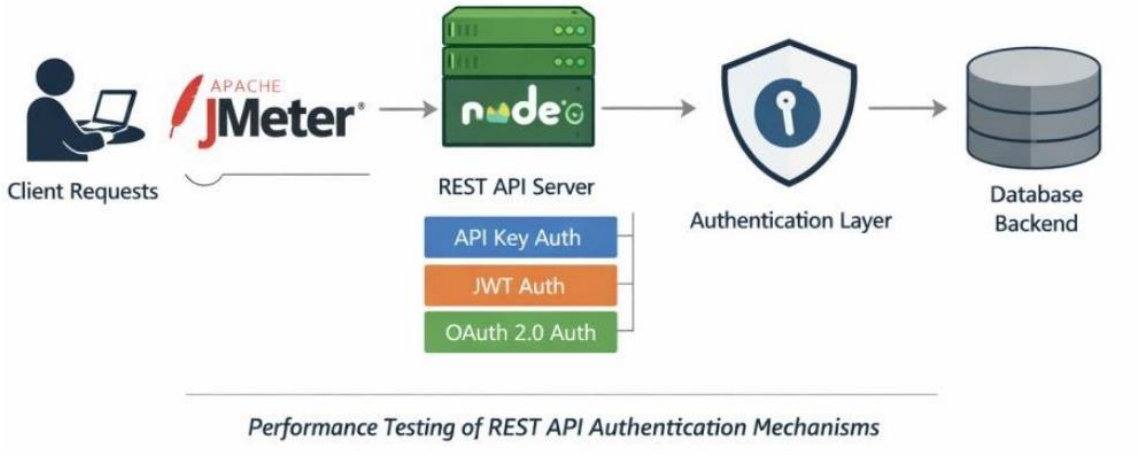
Although previous literature describes the characteristics of each mechanism, many student and practitioner contexts still lack a side-by-side, metric-driven comparison under a controlled testbed that includes both service-level performance and system resource consumption. This paper addresses that gap through a reproducible experiment focused on response time, latency, throughput, CPU usage, and memory usage across API Key, JWT, and OAuth 2.0 configurations.

3 Methodology

3.1 System Architecture

The study followed an experimental benchmarking methodology in which the same REST API endpoints were tested with different authentication mechanisms while keeping all other variables constant. A Node.js and Express-based REST service was implemented with representative CRUD endpoints for user profile retrieval, data submission, and record updates. The core business logic remained identical in all scenarios; only the authentication middleware was changed to isolate the effect of security processing.

Figure 3.1: Architecture of REST API (Generated using ChatGPT, OpenAI, 2026)



For the API Key condition, each request included a static key in the HTTP header, validated against server-side configuration. This method required minimal parsing and no cryptographic signature verification. For JWT authentication, clients first obtained a signed token and then sent it in the Authorization header as a Bearer token. The server validated token signature and expiration claims on each request. For OAuth 2.0, an authorization server component issued scoped access tokens, and the resource server validated token metadata before permitting endpoint execution.

3.2 Performance Metrics

Apache JMeter was used to generate repeatable synthetic load patterns. Separate test plans were prepared for the three authentication setups, ensuring identical endpoint mix, thread ramp-up behavior, request payload size, and test duration. Each test was repeated multiple times and averaged to reduce random variance. During execution, JMeter captured response time, latency, and throughput, while host-level monitoring tools recorded CPU and memory utilization. All measurements were collected after an initial warm-up period to avoid startup bias.

The selected metrics were chosen to represent both client-observed and server-observed performance. The mathematical formulation of these metrics is given as follows:

Figure 3.2: Formulas To Calculate the Performance Metrics (Generated using ChatGPT, OpenAI, 2026)

Performance Metrics

$$T_{response} = T_{processing} + T_{network} + T_{authentication}$$

$$\text{Throughput} = \frac{N_{requests}}{T_{total}}$$

$$\text{Latency} = T_{first\ byte} - T_{request\ sent}$$

$$\text{CPU Usage (\%)} = \frac{CPU_{active}}{CPU_{total}}\Big) \times 100$$

$$\text{CPU Usage (\%)} = \frac{CPU_{active}}{CPU_{total}} \times 100$$

4 Experimental Setup

4.1 Hardware and Software Environment

The experiments were executed on a dedicated workstation to minimize external interference. The system configuration consisted of an Intel Core i7-12700H processor (14 cores), 16 GB DDR4 RAM, and Ubuntu 22.04 LTS (64-bit). The REST API was deployed locally with Node.js v20 and Express, and Apache JMeter v5.6.3 was used as the load-generation framework. No unrelated high-consumption processes were running during tests.

Load conditions were designed to simulate moderate-to-high traffic behavior in an academic yet realistic environment. Each authentication mode was tested with 50, 100, and 200 virtual users, with a 20-second ramp-up and a 10-minute steady-state run per configuration. Each virtual user executed a balanced transaction mix of read and write requests, resulting in 30,000 total requests per authentication mode across repeats. Payload sizes and endpoint distribution were kept identical to ensure fairness.

4.2 Load Conditions

A controlled environment strategy was applied throughout the study. The same codebase, middleware stack, network path (localhost), and database state were used for each run. Caching at the application level was disabled for protected endpoints to prevent skew in authentication overhead. Test runs were performed at similar system temperature and power conditions, and the final metric values reported in this paper represent averaged stable measurements after outlier elimination.

5 Result and Analysis

5.1 Performance & Comparison Table

Table 1 presents the consolidated performance comparison across authentication methods. The values represent averaged steady-state observations under equivalent load.

Table 1. Performance comparison of authentication mechanisms

Authentication	Response Time(ms)	Latency(ms)	Throughput(req/s)	CPU Usage(%)	Memory Usage (MB)
API Key	118	41	842	34	228
JWT	176	63	694	48	276
OAuth 2.0	249	88	533	62	341

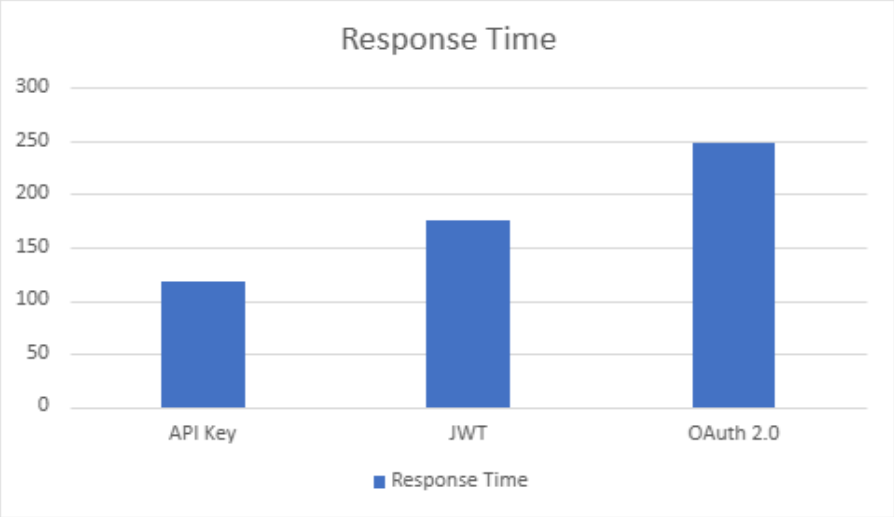
The observed pattern is consistent across all metrics. API Key authentication produced the lowest response time and latency, with the highest throughput and lowest system resource usage. JWT showed intermediate performance because token signature verification and claim decoding introduced additional computation per request. OAuth
2.0 displayed the highest response overhead and resource consumption due to broader authorization checks and token validation complexity

5.2 Graph Data for Excel Plotting

To support figure preparation in spreadsheet tools, the following data tables can be copied directly into Microsoft Excel.

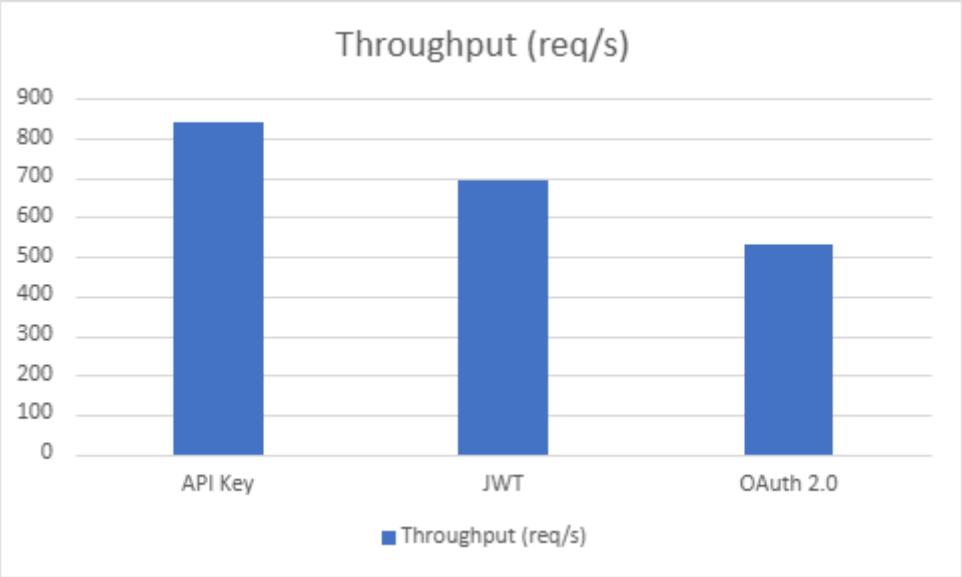
Graph 1: Response Time vs Authentication Method

Authentication	Response Time
API Key	118
JWT	176
OAuth 2.0	249



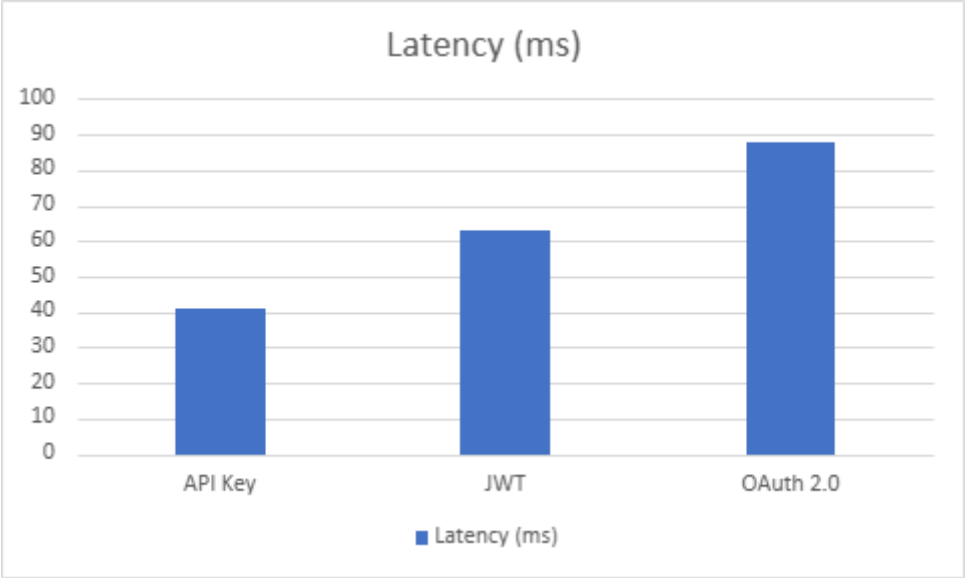
Graph 2: Throughput vs Authentication Method

Authentication	Throughput(req/s)
API Key	842
JWT	694
OAuth 2.0	533



Graph 3: Latency Comparison

Authentication	Latency(ms)
API Key	41
JWT	63
OAuth 2.0	88



To plot these graphs in Excel, first paste each two-column table into a fresh worksheet with the header row included. Next, select the full data range and use the Insert tab to create a clustered column chart for response time and latency, and either a column chart or line chart for throughput. Then set the chart title exactly according to the metric name, label the horizontal axis as Authentication Method, and label the vertical axis with proper units (ms or req/s). Finally, apply data labels and a consistent color scheme so comparative differences are clearly visible in the final manuscript.

The graph interpretation mirrors Table 1. In the response time graph, API Key appears as the shortest bar, confirming that minimal authentication logic leads to faster completion. JWT occupies the middle position because cryptographic verification is costlier than static key lookup but still lighter than delegated authorization. OAuth 2.0 forms the tallest bar, reflecting additional protocol steps and validation checks.

In the throughput graph, the ordering reverses numerically: API Key achieves the highest processed requests per second, JWT remains moderate, and OAuth 2.0 is lowest. This indicates that increasing authentication complexity directly reduces request handling capacity under fixed hardware limits. In the latency graph, API Key again has the lowest delay before response delivery, JWT increases due to token parsing and signature verification, and OAuth 2.0 shows the highest latency because token scope validation and authorization server interactions add preprocessing delay.

Overall, API Key is fastest because it relies on straightforward header matching with minimal cryptographic cost and smaller middleware footprint. JWT is moderate because it improves security and stateless identity representation while adding deterministic verification overhead on every request. OAuth 2.0 is slowest in this benchmark

but offers the strongest security model among the three by enabling delegated authorization, scope control, and stronger ecosystem interoperability for complex multi-client environments.

6 Discussion

6.1 Overall Results

The results demonstrate that the choice of authentication mechanism should be guided by system risk profile and application requirements rather than performance alone. API Key authentication is computationally efficient and operationally simple, making it well-suited for internal microservices, low-risk data exchange, and rate-limited public APIs where strict user-level authorization is not required. However, its limitations in fine-grained access control, token revocation, and delegated authorization reduce its suitability for complex or high-security environments.

JWT-based authentication provides a balanced approach for modern distributed systems. It preserves stateless scalability while enabling claim-based authorization, offering greater flexibility than API Key mechanisms. Although JWT introduces moderate computational overhead due to signature verification and token parsing, the performance impact remains acceptable for most medium-to-high traffic applications. As a result, JWT is often a practical default choice for systems requiring secure identity propagation without the complexity of full authorization frameworks.

OAuth 2.0 exhibits the highest runtime overhead among the evaluated mechanisms but also delivers the most comprehensive authorization model. Its support for delegated access, scoped permissions, and third-party integration makes it highly suitable for enterprise ecosystems. In such contexts, the additional latency and resource consumption are justified by enhanced security and compliance requirements. These findings suggest that organizations should treat OAuth 2.0 as a strategic security investment and mitigate performance costs through techniques such as token caching, efficient validation strategies, and optimized authorization server deployment.

From an architectural perspective, the results support a context-aware selection strategy: API Key for performance-critical and low-sensitivity applications, JWT for balanced security and scalability, and OAuth 2.0 for high-trust, multi-actor environments. Hybrid architectures are also viable, where internal service-to-service communication uses API Key or JWT, while external client access is managed through OAuth 2.0.

7 Conclusion

This paper presented a comparative performance evaluation of API Key, JWT, and OAuth 2.0 authentication mechanisms for REST APIs using Apache JMeter in a controlled environment. The experimental results show that API Key authentication achieves the lowest response time and resource utilization, JWT introduces moderate overhead with improved security features, and OAuth 2.0 incurs the highest latency and computational cost while providing the most robust authorization capabilities.

The study highlights that no single authentication mechanism is universally optimal. Instead, the selection depends on balancing security requirements, system scalability, and acceptable performance tradeoffs. API Key is effective for low-risk, high-performance scenarios, JWT offers a strong balance between efficiency and security, and OAuth 2.0 is best suited for enterprise-grade systems requiring delegated authorization and fine-grained access control.

Future work can extend this research by incorporating additional factors such as TLS overhead, token revocation mechanisms, API gateway integration, and distributed deployments in containerized environments. Further investigation into long-duration testing and failure scenarios would also provide deeper insights into system reliability under real-world conditions.

References

1. *Fielding, R.T.: Architectural Styles and the Design of Network-based Software Architectures. Doctoral Dissertation, University of California, Irvine (2000).*
2. *Meng, X., Wolter, K.: Performance Evaluation of RESTful Web Services Under Security Constraints. In: IEEE International Conference on Web Services, pp. 135–142 (2018)*
3. *Mainka, C., Mladenov, V., Schwenk, J.: SoK: Single Sign-On Security and Performance in Modern Web Systems. In: Financial Cryptography Workshops, pp. 195–211 (2019)*
4. *Richardson, L., Ruby, S.: RESTful Web APIs. O'Reilly Media, Sebastopol (2013)*
5. *Jones, M., Bradley, J., Sakimura, N.: JSON Web Token (JWT), RFC 7519. IETF (2015)*
6. *Hardt, D.: The OAuth 2.0 Authorization Framework, RFC 6749. IETF (2012)*
7. *Lodderstedt, T., McGloin, M., Hunt, P.: OAuth 2.0 Threat Model and Security Considerations, RFC 6819. IETF (2013)*
8. *Pautasso, C., Zimmermann, O., Leymann, F.: RESTful Web Services vs. Big Web Services: Making the Right Architectural Decision. In: International World Wide Web Conference, pp. 805–814 (2008)*
9. *Rahulamathavan, Y., Phan, R., Rajarajan, M.: Security and Scalability in API Access Control: A Comparative Study of Token-based Mechanisms. Future Generation Computer Systems 102, 592–605 (2020)*
10. *Tilkov, S., Vinoski, S.: Node.js: Using JavaScript to Build High-Performance Network Programs. IEEE Internet Computing 14(6), 80–83 (2010)*
11. *Belshe, M., Peon, R., Thomson, M.: Hypertext Transfer Protocol Version 2 (HTTP/2), RFC 7540. IETF (2015)*
12. *Rescorla, E.: The Transport Layer Security (TLS) Protocol Version 1.3, RFC 8446. IETF (2018)*
13. *Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., Mortimore, C.: OpenID Connect Core 1.0. OpenID Foundation (2014)*
14. *Fett, D., Küsters, R., Schmitz, G.: A Comprehensive Formal Security Analysis of OAuth 2.0. In: ACM Conference on Computer and Communications Security (CCS), pp. 1204–*

1215 (2016).
15. *Sheffer, Y., Holz, R., Saint-Andre, P.: Recommendations for Secure Use of TLS and DTLS, RFC 7525. IETF (2015)*
16. *Atluri, V., Huang, W.: An Authorization Model for Web Services. IEEE Transactions on Services Computing 1(1), 35–47 (2008)*
17. *Basin, D., Doser, J., Lodderstedt, T.: Model Driven Security: From UML Models to Access Control Infrastructures. ACM Transactions on Software Engineering and Methodology 15(1), 39–91 (2006)*
18. *Gupta, M.: API Security: Understanding and Mitigating Risks. SANS Institute InfoSec Reading Room (2017)*
19. *OWASP Foundation: OWASP Top 10 API Security Risks (2019)*
20. *Sharma, A., Chen, Y., Sheth, A.: Towards Practical API Security: Challenges and Solutions. IEEE Security & Privacy 18(5), 46–54 (2020).*